

Down with typename!

Summary

If $x < T > :: Y$ — where T is a template parameter — is to denote a type, it must be preceded by the keyword `typename`; otherwise, it is assumed to denote a name producing an expression. There are currently two notable exceptions to this rule: *base-specifiers* and *mem-initializer-ids*. For example:

```
template<class T>
struct D: T::B { // No typename required here.
};
```

Clearly, no `typename` is needed for this *base-specifier* because nothing but a type is possible in that context. However, there are several other places where we know only a type is possible and asking programmers to nonetheless specify the *typename* keyword feels like a waste of source code space (and is detrimental to readability).

We therefore propose we make `typename` optional in a number of commonplace contexts that are known to only permit type names.

A cursory read through some common standard library headers suggests that by-far most occurrences of `typename` for the purpose of disambiguating type names from other names can be eliminated with the new rules.

The EDG front end has an ‘implicit `typename`’ mode to emulate pre-C++98 compilers that didn't parse templates in their generic form. Although that mode doesn't exactly cover the contexts where we are proposing to make `typename` optional, the implementation effort is similar (and not excessively expensive).

This proposal was approved in Toronto (July 2017) by EWG:

SF: 14 | F: 8 | N: 1 | A: 0 | SA: 0

Changes since P0634R1

- HTML angle brackets typo fix
- Merged with changes for CWG issue 1271: Imprecise wording regarding dependent types
- addressed review comments

Wording changes

☐ Hide deleted text

[Change in 17.7 \[temp.res\] paragraphs 3 and 4:](#)

~~A *qualified-id* is intended to refer to a type that is not a member of the current instantiation (17.6.2.1) and its *nested-name-specifier* refers to a dependent type, it shall be prefixed by the keyword `typename`, forming a *typename-specifier*. If the *qualified-id* in a *typename-specifier* does not denote a type or a class template, the program is ill-formed.~~

```
typename-specifier:
    typename nested-name-specifier identifier
    typename nested-name-specifier templateopt simple-template-id
```

A *typename-specifier* denotes the [type or class template denoted by the simple-type-specifier \(10.1.7.2 \[dcl.type.simple\]\)](#) formed by omitting the keyword `typename`.

~~If a specialization of a template is instantiated for a set of *template-arguments* such that the *qualified-id* prefixed by~~

typename does not denote a type or a class template, the specialization is ill-formed. The usual qualified name lookup (6.4.3) is used to find the qualified-id even in the presence of typename. [Example: ...]

Add a new paragraph in [temp.res] before paragraph 6:

A *qualified-id* is assumed to name a type if

- it is a qualified name in a *type-id-only context* (see below), or
- it is the *decl-specifier-seq* of a
 - *simple-declaration* or a *function-definition* in namespace scope,
 - *member-declaration*,
 - *parameter-declaration* in a *member-declaration*, unless that *parameter-declaration* appears in a default argument [Note: This includes friend function declarations. —end note],
 - *parameter-declaration* in a *declarator* of a *function-definition* for a class-member, unless that *parameter-declaration* appears in a default argument,
 - *parameter-declaration* of a function parameter appearing in a *lambda-declarator*, or
 - *parameter-declaration* of a (non-type) *template-parameter*.

A qualified name is said to be in a *type-id-only context* if it appears in a *type-id*, *new-type-id*, or *defining-type-id* and the smallest enclosing *type-id*, *new-type-id*, or *defining-type-id* is a

- *new-type-id*,
- *defining-type-id*,
- *trailing-return-type*,
- default argument of a *type-parameter* of a template, or
- *type-id* of a *static_cast*, *const_cast*, *reinterpret_cast*, or *dynamic_cast*.

[Example:

```
template<class T> T::R f(); // OK, return type of a function declaration at global scope
template<class T> void f(T::R); // Error: parameter of a function declaration at global scope
template<class T> struct S {
    using Ptr = PtrTraits<T>::Ptr; // OK, in a defining-type-id
    T::R f(T::P p) { // OK, class scope
        return static_cast<T::R>(p); // OK, type-id of a static_cast
    }
    auto g() -> S<T*>::Ptr; // OK, trailing-return-type
};
template<typename T> void f() {
    void (*pf)(T::X); // Variable pf of type void* initialized with T::X
    void g(T::X); // Error: T::X at block scope does not denote a type
}
```

— end example]

Change in 17.7 [temp.res] paragraph 6:

If, for a given set of template arguments, a specialization of a template is instantiated that refers to a qualified-id that denotes a type or a class template, and the qualified-id refers to a member of an unknown specialization, the qualified-id shall either be prefixed by typename or shall be used in a context in which it implicitly names a type as described above. A *qualified-id* that refers to a member of an unknown specialization, **that** is not prefixed by typename, and **that** is not otherwise assumed to name a type (see above) denotes a non-type. [Example:

```
template <class T> void f(int i) {
    T::x * i; // T::x must not be a type expression, not the declaration of a variable i
}
struct Foo {
    typedef int x;
};
struct Bar {
    static int const x = 5;
};
int main() {
    f<Bar>(1); // OK
    f<Foo>(1); // error: Foo::x is a type
}
```

-- end example]

Change in 17.7 [temp.res] paragraphs 7 and 8:

Within the definition of a class template or within the definition of a member of a class template following the *declarator-id*, the keyword `typename` is not required when referring to a member of the current instantiation (17.7.2.1 [temp.dep.type]). the name of a previously declared member of the class template that declares a type or a class template. [Note: Such names can be found using unqualified name lookup (6.4.1), class member lookup (6.4.3.1) into the current instantiation (17.7.2.1), or class member access expression lookup (6.4.5) when the type of the object expression is the current instantiation (17.7.2.2). -- end note] [Example:

```
template<class T> struct A {  
    typedef int B;  
    B b;           // OK, no typename required  
};
```

-- end example]

[Note: Knowing which names are type names allows the syntax of every template to be checked. -- end note] The program is ill-formed, no diagnostic required, if: ...